CEN 3016 SPECIFICATIONS OF SOFTWARE SYSTEMS

Clean Architecture

Chapters 16–20

Independence, boundaries, policy and business rules

MSc. Jose Ayerdis

Lecture sequence

Chapter	Topic	Guiding question
16	Independence	What should change independently?
17	Boundaries: Drawing Lines	Where should responsibilities separate?
18	Boundary Anatomy	How do components cross a boundary?
19	Policy and Level	Which decisions are higher-level policy?
20	Business Rules	Where do entities and use cases belong?

Learning outcomes

01 Explain independence and justify an architectural boundary.

02 Describe a crossing and the interface that controls it.

03 Distinguish policy level from technical importance.

04 Separate entity rules from use-case coordination.

APPLICATION Use these distinctions to explain and defend a small system model.

Independence

Architecture must accommodate several kinds of change

Concern	What architecture should support
Use cases	The system’s intended behaviors
Operation	Runtime needs such as capacity and responsiveness
Development	Teams changing responsibilities with limited interference
Deployment	Building, releasing and starting components predictably

QUESTION Which concerns can pull the structure in different directions?

Decoupling layers and use cases

BY RESPONSIBILITY

Presentation handles interaction.
Application logic coordinates goals.
Persistence handles storage.

BY USER GOAL

Borrow a copy.
Return a copy.
Renew a loan.

A change to one layer or goal should have a limited, explainable impact.

Modes of decoupling

Mode	Separation	Consideration
Source level	Modules and interfaces	One application can preserve logical boundaries
Deployment level	Separate deployable components	Contracts and version compatibility matter
Service level	Communication across service boundaries	Network behavior adds operational concerns

Choose a mode for the needs you can justify. Keep later options practical.

Options and duplication

OPTIONS

Delay a vendor commitment when a useful interface can isolate it.

Revise the structure as evidence about operations and teams grows.

DUPLICATION

Similar code may serve policies that change for different reasons.

Sharing that code can couple otherwise independent use cases.

CHECK What evidence shows that two pieces of code represent the same rule?

Concept check: independence

A team adds a web interface to an existing application.

01 Which business rules should stay the same?

02 Which components are likely to change?

03 Would a separate service be required? Explain.

Boundaries: drawing lines

A boundary separates responsibilities that should evolve independently.

Policy expresses a decision or rule.
A mechanism supplies a way to carry it out.

QUESTION Which changes should remain on one side of the line?

Policy and replaceable mechanisms

Conceptual dependencies in a lending application



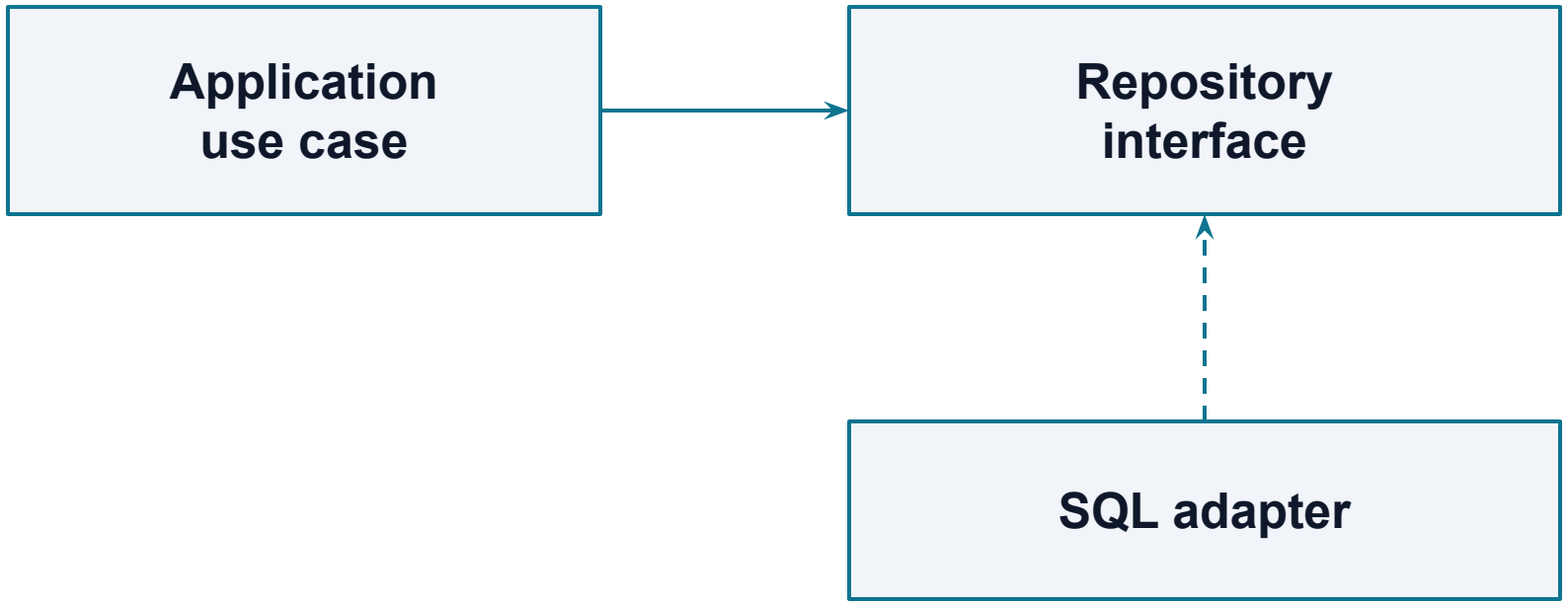
Shows information

Owns rules and contracts

Implements storage needs

Arrows show source dependencies toward policy. Interface details come next.

A plugin relationship



Application owns the interface.

Adapter implements the contract.

Which decisions belong behind a boundary?

01 Database: query syntax and persistence mapping.

02 Interface: screen layout, routes and form fields.

03 Provider: vendor-specific messages and credentials.

04 Core: rules and the contracts that express its needs.

DECISION Name the change you want to isolate before drawing the boundary.

Concept check: a boundary inside one app

One executable contains interface, loan rules and persistence code.

Where would you draw a line?
What contract would it expose?
What change would that line isolate?

Boundary anatomy

01 Components hold responsibilities on each side.

02 A contract defines what crosses the boundary.

03 Calls or messages carry requests and results.

04 Source dependencies determine what code knows about what.

QUESTION Can a call cross a boundary without a network request?

Physical forms of a boundary

Form	How interaction occurs	What to consider
One executable	Function or method call	Logical dependency control
Deployable components	Component interfaces	Packaging and compatibility
Local processes	Interprocess communication	Serialization and process lifecycle
Remote services	Network messages	Latency, timeouts and partial failure

Threads describe execution. They do not automatically define architectural boundaries.

Source dependency and runtime call

SOURCE DEPENDENCIES



Application declares the contract. The adapter implements it.

At runtime, the use case calls the injected adapter through that interface.

The contract at a crossing

01 Input: the values and identity needed for a request.

02 Output: a result the receiving side can interpret.

03 Failure: rejection, timeout or unavailable dependency.

04 Ownership: which side declares and evolves the contract?

DISCUSS What becomes harder when an in-process call moves across a network?

Concept check: boundary anatomy

01 Does a boundary require separate deployment?

02 Can runtime control and source dependency point differently?

03 What does a caller need when a dependency fails?

EXPLAIN Use one component pair and describe its contract.

Policy and level

POLICY

A decision about behavior or transformation.

Example: determine whether a member may borrow a copy.

LEVEL

A policy's abstraction relative to input/output details.

Business meaning sits farther from device-specific operations.

DISTINCTION Operational importance and policy level describe different things.

Levels of policy and detail

Responsibility	Example	Reason for change
General domain policy	A loan has an allowed due date	Lending policy changes
Application policy	Coordinate eligibility, checkout and saving	The checkout workflow changes
Input/output detail	Parse a web form or map a SQL row	The interface or storage changes

Dependencies should protect higher-level meaning from lower-level changes.

Dependencies toward higher-level policy



Closer to external input

Application behavior

General domain meaning

Source dependencies, rather than the runtime sequence, define these arrows.

Concept check: classify the responsibility

01 A member may hold at most three active loans.

02 Checkout verifies eligibility and records a new loan.

03 A controller reads a member ID from a web request.

04 A persistence adapter writes a row using SQL.

JUSTIFY Which decisions are policy, and which are technical details?

Business rules and entities

BUSINESS RULES

Rules that make the business behavior correct.

They retain meaning even when people perform the work manually.

ENTITIES

Encapsulate critical business data and the rules that govern it.

They can be objects or suitable data structures and functions.

EXAMPLE Which rules would a lending desk still follow without an app?

Entities and use cases

Aspect	Entity	Use case
Responsibility	Protect general domain rules	Coordinate an application goal
Example	Loan and its valid state	Check out a copy
Change trigger	Domain policy changes	Application workflow changes
Dependency	Independent of use-case workflow	Uses entities and application-owned contracts

The use case applies entity rules to a particular request.

A use-case specification

Example: Check out a copy

ACTOR AND CONDITIONS

Member requests a copy.
An identified member and a copy ID accompany the request.

FLOW AND OUTCOME

Check eligibility and availability.
Ask domain behavior to create a loan.
Save the result and return the due date.

If a rule fails, explain the rejection.

Entity invariant: one physical copy cannot have two active loans.

Request and response models

APPLICATION INPUT

CheckoutRequest

memberId
copyId

APPLICATION RESULT

CheckoutResult

loanId and dueDate
or a defined rejection reason

Simple application-owned data keeps external formats behind a boundary.

The concepts working together

Chapter	Design question	Evidence
16	What should change independently?	One justified change scenario
17	Where should we draw a line?	Responsibilities on each side
18	How is the line crossed?	Contract, interaction and failure result
19	Which policy is higher level?	Classified responsibilities and dependency arrows
20	Who owns each business rule?	Entities, use cases and acceptance examples

Reading and reference material

ASSIGNED READING

Robert C. Martin, Clean Architecture
16 Independence
17 Boundaries: Drawing Lines
18 Boundary Anatomy
19 Policy and Level
20 Business Rules

Course syllabus: detailed calendar, page 8. Full links appear in speaker notes.

Application: a PedidosYa ordering scenario

Ana orders lunch in Managua. The restaurant receives the request, then a courier delivers it.

CLASSROOM ASSUMPTIONS

One restaurant per order. Payment on delivery.
The customer confirms the total before submission.
Only Submitted orders allow ordinary customer cancellation.
Agreed prices stay with the order after submission.

Simplified teaching rules, rather than verified PedidosYa policies

A brief connection to all five chapters

Chapter	Apply it to this scenario
16 Independence	Add a web interface while keeping ordering rules
17 Boundaries	Separate ordering behavior from interface and storage
18 Boundary Anatomy	Describe a cancellation request crossing those boundaries
19 Policy and Level	Distinguish cancellation policy from HTTP and SQL
20 Business Rules	Model Order, its rules and the Cancel order use case

Group activity: model Cancel order

Teams of 3–4. Use draw.io or a shared whiteboard.

01 5 minutes: agree on scope, entities and assumptions.

02 10 minutes: draw boundaries and define the crossings.

03 10 minutes: classify policy and specify business rules.

04 5 minutes: test the model against one change.

Output: one annotated model plus a short rule table and use-case description.

Tasks: boundaries and boundary anatomy

CHAPTER 17

Draw the responsibilities on each side of each boundary.

Identify the change each line should isolate.

CHAPTER 18

Choose how components interact. Name the interface owner.

Specify request, result and one failure outcome.

Show source dependencies. Explain runtime control separately.

Tasks: policy level and business rules

CHAPTER 19

Classify domain policy, application policy and I/O details.

Justify the direction of each source dependency.

CHAPTER 20

Identify entities and their rules. Specify Cancel order.

Write three rule statements and two acceptance examples.

Each rule should define a condition and an observable consequence.

Change and conflict checks

Choose one check and trace its consequences through the model

Check	What your team must explain
New interface	Can the web app reuse the same policy and use case?
New database	Which adapter changes, and which contract remains?
New cancellation policy	What changes if cancellation is allowed after acceptance?
Simultaneous requests	Can restaurant acceptance and cancellation both commit?

Group share-out and review

Each team has 2–3 minutes to defend its model

01 Explain one boundary and the change it isolates.

02 Trace one request across an owned contract.

03 Justify one policy-level and dependency decision.

04 Demonstrate one business rule with success and rejection.

EXIT CHECK Which assumption most needs confirmation from a real stakeholder?